

Document Number: P1091R1

Date: 2018-10-08

Reply-to: Nicolas Lesser <blitzrakete@gmail.com>

Audience: EWG

Extending structured bindings to be more like variable declarations

Changes

The following things were changed to R0:

- Add lambda captures
- Add structured binding declaration templates
- Add `[[maybe_unused]]`
- Better wording

Introduction

There are a lot of restriction on structured bindings compared to variable declarations, like not being able to mark them `static`, `constexpr` or them not having unclear linkage. This proposal's aim is to fix this by for example making the underlying structured binding object (and tuple binding variables) have external linkage and by allowing various specifiers (`static`, `thread_local`, `constexpr`, `inline`, `extern`) on structured bindings.

Motivation

Structured bindings, although very useful, are actually pretty magical. They don't introduce variables in the normal sense for each binding, rather, they are names that refer to specific objects. As such, there are problems including what it means for a binding to be `static` and how it would work and what linkage do those bindings even have.

Those problems could not be resolved during the discussion of the paper and afterwards, a paper was requested to analyse the possible design impact that such additions to structured bindings would have after two NB comments proposing this were rejected. This paper attempts to do so.

One motivation to do so is to bring structured bindings closer to actual variable declarations, so consistency. This will also make structured bindings more useful, as they are currently lacking for example `constexpr`, which is becoming every more important for various features of the language.

As a consequence this paper also fixes some DRs that were filed and are under consideration or going to be eventually by either Evolution or Core.

Linkage

As per `[basic.link]p8`, bindings do not have any linkage because they're just names. Currently, in both gcc and clang, tuple binding have linkage, as they were specified as variable declarations in the standard. This is no longer the case though due to the resolution of DR2313¹.

But the underlying structured binding object is an actual variable, which can have either internal or external linkage depending on the declaration of the structured binding, as gcc and clang do it. There is no way to refer to

that object though without making the program IF-NDR as the object has a name like `_ZDC1a2bbE` for `auto[a, bb]`.

`[dcl.struct.bind]p1` has the following to say about the object:

First, a variable with a unique name *e* is introduced.

It follows that the variable cannot be referenced in a conforming program anyways, and as such, it doesn't make much sense to give it external linkage. Nonetheless, to be consistent with the rest of the declarations and being able to use just `inline` (see below) without an extra `extern` to give the structured binding external linkage, the underlying object should have external linkage.

extern

As discussed in the previous section, there is no way to reference either the bindings or the underlying object within a conforming program, so allowing `extern` on such a structured binding would not make much sense.

However, this would prohibit `inline` on structured bindings that have been declared `const` and `inline`, which might be desirable in some cases. For this reason, it should be allowed. Note that `extern` would have no effect on the individual bindings, except for the tuple case.

static and thread_local

`static` and `thread_local` on a structured binding make sense and are actually useful. The way to make this work nicely in the standard is to only apply them on the underlying object, and not on the bindings (which wouldn't make sense and can't work today without major changes to the specification of bindings anyways). For the tuple case on the additional variable declarations too.

Because bindings refer to certain objects (depending on the initializer of the structured binding), they would implicitly get the desired semantics of `static` and `thread_local`, as they refer to either objects within the underlying object or to separate variables (in the tuple case) which are marked with the desired specifiers.

inline

`inline` is also useful and will be consistent with inline variables. It will work just as with `static` and `thread_local`: The underlying object is marked `inline` and any additional variables introduced as part of tuple bindings.

constexpr

If `constexpr` were applied just like `static` and `co_` are, then there would be a problem, because the current language rules make the following code ill-formed:

```
// at block scope
constexpr auto[a] = std::tuple<int>(1);
// "equivalent" to
constexpr auto __sb = std::tuple<int>(1);
constexpr const int& __a = std::get<0>(__sb); // ill-formed today
```

A reference must be initialized by a constant expression to be a core constant expression ([`expr.const`]p2.11], but `std::get<0>(__sb)` is not a constant expression due to [`expr.const`]p6.

Richard Smith on the core reflector² suggested to relax the restriction on what constitutes a core constant expression of a reference by just requiring that the reference must be initialized by a *core* constant expression instead (see below to what this change entails).

This would mean that to make structured bindings `constexpr`, it is necessary to apply `constexpr` to the underlying object, and apply `const` to any other variable introduced by tuple bindings.

Of course, one thing to note is that it is important to guarantee that the call to `get` is a constant expression, because or else `constexpr` will act like `const`, which is only maybe a core constant expression.

lambda captures

Lambda captures aren't currently allowed to refer to a structured binding. There seems to be no technical reason to disallow this, and indeed, the wording for allowing this just removes the restriction on capturing structured bindings.

[[maybe_unused]]

Currently, `[[maybe_unused]]` cannot be applied to a structured binding declaration. There doesn't seem to be a good reason to disallow this; EDG, clang and gcc all already support `[[maybe_unused]]` on structured bindings.

Template

The following code is ill-formed as of C++20:

```
template <auto Var>
constexpr auto[X, Y] = Var;
```

As it uses 1) a structured binding declaration template and 2) it is `constexpr`. The latter is already handled. Should a structured binding be a valid *template-declaration*? The author argues that yes, it should be. It allows for code that can decompose any non-type template parameter (which now can be any class type, thanks to P0732r2).

```
constexpr std::pair Position(1, 2);
constexpr std::pair Flag(4, 5);

if (X<Position> == Y<Flag>)
    ; // almost there!
```

Note: The author doesn't know how to change the standard to allow this completely.

Impact

This proposal only makes ill-formed or code with unspecified behavior well-formed in relation to structured bindings.

Due to `constexpr` tuple binding variables requiring a change to what constitutes a core constant expression, ill-formed code today will become well-formed:

```
// at block scope

const int var = 1;
```

```
const int& ref = var;  
static_assert(ref == 1); // ill-formed today, well-formed with this proposal
```

Proposed wording

All changes relative to the latest C++20 draft.

Change [basic.link]p10 (6.5) as follows:

Two names that are the same and that are declared in different scopes shall denote the same variable, function, type, template or namespace if

- they denote variables introduced by structured binding declarations formed with the same sequence of identifiers, or

Change [expr.prim.lambda.capture]p8 (7.5.5.2) as follows:

If a *lambda-expression* explicitly captures an entity that is not odr-usable ~~or captures a structured binding (explicitly or implicitly)~~, the program is ill-formed.

Change [expr.const]p2 (8.6) as follows:

- an *id-expression* that refers to a variable or data member of reference type unless the reference has a preceding initialization and either
 - it is initialized with a core constant expression or
 - its lifetime began within the evaluation of *e*;

Change [dcl.attr.unused]p2 (9.11.8) as follows:

The attribute may be applied to the declaration of a class, a *typedef-name*, a variable, a structured binding declaration, a non-static data member, a function, an enumeration, or an enumerator.

Change [dcl.dcl]p8 (10) as follows:

A *simple-declaration* with an *identifier-list* is called a *structured binding declaration* ([dcl.struct.bind]). The *decl-specifier-seq* shall contain only constexpr, inline, the *type-specifier* *auto*, the storage-class-specifiers *static* and *thread_local*, and *cv-qualifiers*. The decl-specifier-seq shall only contain the storage-class-specifier *extern* if the simple-declaration is not a declaration-statement.

Change [dcl.stc]p3 (10.1.1) as follows:

The *thread_local* specifier indicates that the named entity has thread storage duration. It shall be applied only to the names of variables of namespace or block scope, to structured binding declarations ([dcl.struct.bind]), and to the names of static data members.

Change [dcl.stc]p4 (10.1.1) as follows:

The *static* specifier can be applied only to names of variables and functions, to structured binding declarations ([dcl.struct.bind]), and to anonymous unions.

Change [dcl.stc]p5 (10.1.1) as follows:

The *extern* specifier can be applied only to the names of variables and functions and to structured binding declarations ([`dcl.struct.bind`]).

Change [`dcl.constexpr`]p1 (10.1.5) as follows:

The *constexpr* specifier shall be applied only to the definition of a variable or variable template or the declaration of a function or function template, or to a structured binding declaration ([`dcl.struct.bind`]).

Change [`dcl.inline`]p1 (10.1.6) as follows:

The *inline* specifier can be applied only to the declaration or definition of a variable or function, or to a structured binding declaration ([`dcl.struct.bind`]).

Change [`class.copy.elision`]p1 (10.9.5) as follows:

This elision of copy/move operations, called copy elision, is permitted in the following circumstances (which may be combined to eliminate multiple copies):

- ~~in a return statement in a function with a class return type, when the expression is the name of a non-volatile automatic object (other than a function parameter or a variable introduced by the exception-declaration of a handler ([`except.handle`])) with the same type (ignoring cv-qualification) as the function return type, the copy/move operation can be omitted by constructing the automatic object directly into the function call's return object~~
- ~~in a throw-expression, when the operand is the name of a non-volatile automatic object (other than a function or catch-clause parameter) whose scope does not extend beyond the end of the innermost enclosing try-block (if there is one), the copy/move operation from the operand to the exception object ([`except.throw`]) can be omitted by constructing the automatic object directly into the exception object~~
- when the exception-declaration of an exception handler ([`except`]) declares an object of the same type (except for cv-qualification) as the exception object ([`except.throw`]), the copy operation can be omitted by treating the exception-declaration as an alias for the exception object if the meaning of the program will be unchanged except for the execution of constructors and destructors for the object declared by the exception-declaration. [Note: There cannot be a move from the exception object because it is always an lvalue. - end note]
- for a structured binding. The rest of this paragraph is followed with the object that the structured binding refers to.
- for an expression *e* who names a non-volatile automatic object (other than a function or catch-clause parameter)
 - in a return statement with a class return type when *e* has the same type (ignoring cv-qualification) as the return type, the copy/move operation can be omitted by constructing the automatic object directly into the function call's return object
 - in a throw-expression when the object (designated by *e*)'s scope does not extend beyond the end of the innermost enclosing try-block (if there is one), the copy/move operation from the operand to the exception object ([`expr.throw`]) can be omitted by constructing the automatic object directly into the exception object

Change [`dcl.struct.bind`]p1 (11.5) as follows:

A structured binding declaration introduces the *identifiers* $v_0, v_1, v_2, \dots$ of the *identifier-list* as names ([`basic.scope.declarative`]) of structured bindings. Let *cv* denote the *cv-qualifiers* in the *decl-specifier-seq*. First, a variable with a ~~unique~~ name *e* is introduced, where *e* is unique except as specified in Clause 6. If the *assignment-expression* in the *initializer* has array type *A* and no *ref-qualifier* is present, ~~*e* has type *cv A*~~ *e* is declared as-if by:

attribute-specifier-seq_{opt} decl-specifier-seq' A e;

where *decl-specifier-seq'* is the *decl-specifier-seq* without the *type-specifier auto*, and each element is copy-initialized or direct-initialized from the corresponding element of the *assignment-expression* as specified by the form of the *initializer*.

Change [dcl.struct.bind]p3 (11.5) as follows:

Otherwise, if the *qualified-id* `std::tuple_size<E>::value` names a complete type, the expression `std::tuple_size<E>::value` shall be a well-formed integral constant expression and the number of elements in the *identifier-list* shall be equal to the value of that expression. The *unqualified-id* *get* is looked up in the scope of *E* by class member access lookup ([basic.lookup.classref]), and if that finds at least one declaration that is a function template whose first template parameter is a non-type parameter, the initializer is `e.get<i>()`. Otherwise, the initializer is `get<i>(e)`, where *get* is looked up in the associated namespaces. In either case, `get<i>` is interpreted as a *template-id*. [Note: Ordinary unqualified lookup is not performed. end note] In either case, *e* is an lvalue if the type of the entity *e* is an lvalue reference and an xvalue otherwise. Given the type T_i designated by `std::tuple_element<i, E>::type`, variables are introduced with unique names r_i ~~as if by: of type "reference to T_i " initialized with the initializer ([dcl.init.ref]), where the reference is an lvalue reference if the initializer is an lvalue and an rvalue reference otherwise.~~

*decl-specifier-seq' cv T_i& r_i = /*initializer*/;*

if the initializer is an lvalue and by:

*decl-specifier-seq' cv T_i&& r_i = /*initializer*/;*

otherwise. *decl-specifier-seq'* refers to the *decl-specifier-seq* of the structured binding declaration except for the *type-specifier auto* and *constexpr*. *cv* is *const* if *constexpr* is in the *decl-specifier-seq*, otherwise it is empty.

Acknowledgements

Thanks to k-ballo, T.C, Richard Smith, Alberto Barbati for mentioning and solving specific problems with this proposal, Jens Maurer for the help in writing the wording, and the countless others on the #future-standard Cpplang channel and the std-proposals mailing list.